

A **systemd Watchdog** forces systemd to automatically restart a Python script if it hangs, enters a deadlock, or freezes in an infinite loop—even if the process is technically still running and hasn't crashed.

The script sends periodic "heartbeat" signals (pings) to systemd. If systemd does not receive a ping within the configured WatchdogSec time window, it kills the unresponsive process with SIGABRT and triggers a restart.

Step 1: Install sd_notify / systemd-python

While you can send watchdog pings manually over UNIX sockets, the lightweight Python package [sd_notify](#) or standard systemd bindings make it seamless.

Install sd_notify via pip:

```
pip install sd_notify
```

Step 2: Implement Watchdog Pings in Python

Use sdnotify to notify systemd that your application is starting up and to periodically send the "WATCHDOG=1" signal inside your main processing loop:

```
#!/usr/bin/env python3
import time
import logging
import sdnotify

logging.basicConfig(level=logging.INFO)

def main():
    # Initialize systemd notifier
    n = sdnotify.SystemdNotifier()

    # Inform systemd that setup is complete and service is ready
    n.notify("READY=1")
    logging.info("Service initialized and marked as READY.")

    while True:
        # Perform normal workload tasks here
        do_work()

        # Send heartbeat ping to systemd
        n.notify("WATCHDOG=1")
        logging.info("Sent watchdog ping to systemd.")

        # Sleep duration MUST be significantly less than WatchdogSec
        time.sleep(5)

def do_work():
    # Example function doing work
    pass
```

```
if __name__ == "__main__":  
    main()
```

Rule of Thumb: Send your watchdog ping at least **2x to 3x as often** as the WatchdogSec interval to avoid false positives caused by temporary system load spikes.

Step 3: Configure WatchdogSec in Systemd Unit File

Add WatchdogSec= to the [Service] section of your unit file
(~/.config/systemd/user/my_watchdog_service.service or
/etc/systemd/system/my_watchdog_service.service).

```
[Unit]  
Description=Python Service with Watchdog Protection  
After=network.target  
  
[Service]  
Type=notify  
WorkingDirectory=%h/projects/my_script_dir  
ExecStart=/usr/bin/python3 -u %h/projects/my_script_dir/main.py  
  
# Watchdog timeout: systemd expects a ping every 15 seconds  
WatchdogSec=15s  
  
# Automatic restart on watchdog failure or crash  
Restart=on-failure  
RestartSec=3s  
  
[Install]  
WantedBy=default.target
```

Note: Change Type=simple to Type=notify so systemd listens for startup READY=1 and watchdog notifications.

Step 4: Step-by-Step Enable & Test Procedure

1. Reload and Start Service

Apply systemd unit changes

Reload the user unit definitions and start your service:

```
systemctl --user daemon-reload  
systemctl --user start my_watchdog_service
```

2. Test Watchdog Triggering

Simulate a deadlock or frozen process

To test if the watchdog kills a frozen process, temporarily inject a freeze into your script:

```
# Simulate a deadlock/hang condition
```

```
if counter > 3:
    logging.warning("Simulating deadlock - stopping watchdog
pings!")
    time.sleep(60) # Exceeds the 15s WatchdogSec limit
```

3. Verify Automated Restart in Logs

Inspect system log output

Check systemd logs with journalctl:

```
journalctl --user -u my_watchdog_service -f
```

When the watchdog timer expires, systemd logs a watchdog timeout and kills the process:

```
my_watchdog_service.service: Watchdog timeout (limit 15s)!
my_watchdog_service.service: Main process exited, code=dumped,
status=6/ABRT
my_watchdog_service.service: Failed with result 'watchdog'.
my_watchdog_service.service: Scheduled restart job, restart
counter is at 1.
```

Pure Python Implementation (No External Libraries)

If you prefer zero third-party dependencies, you can send notifications directly to the systemd socket defined in the NOTIFY_SOCKET environment variable:

```
import os
import socket

def send_systemd_notify(status_msg: str) -> None:
    socket_path = os.environ.get("NOTIFY_SOCKET")
    if not socket_path:
        return

    # Abstract socket addresses start with @
    if socket_path.startswith("@"):
        socket_path = "\0" + socket_path[1:]

    with socket.socket(socket.AF_UNIX, socket.SOCK_DGRAM) as sock:
        sock.sendto(status_msg.encode("utf-8"), socket_path)

# Usage in your loop:
send_systemd_notify("WATCHDOG=1")
```